
WHITE PAPER · PORTFOLIO & AI GOVERNANCE

Automate, Build, Buy, Hire, or Wait

Choosing the response to an AI request

Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

Before funding an AI request, compare five responses: automate, build, buy, hire or train, or wait. The harder a response is to undo, the more proof it needs.

A request that arrives with its response already chosen leaves the other responses uncomparing. This paper gives a three-step rule for choosing among the five, sets how much proof each needs by how hard it is to undo, and shows how to run a wait with an owner, a review date, and a trigger, the event that brings the request back for a decision.

- 01** Drop any response that cannot relieve the step limiting the process it is meant to improve. Send one that needs something not yet in place to wait, and replace one that lacks the proof it needs with an easier-to-undo version. Rank the rest by relief against cost for each year of commitment, and when two are close choose the one that is easier to undo.
- 02** Set the proof by how hard a response is to undo: a reason, a current measurement, and a review date for an easy one, a small test in real work for a costly one, and the full operating case with a written way out for one that is practically permanent.
- 03** Give a wait its own record: a trigger, an owner, a review date, and a stated cost of delay. A wait with no trigger or no owner counts as a stop.

CONTENTS

A request that arrives with its response chosen	A worked example
The five responses	Five ways the choice goes wrong
How to choose among them	Objections, and what the sources do not show
How hard it is to undo sets how much proof a response needs	What to do next
Waiting is a response with a price	Notes and sources

A request that arrives with its response chosen

A steering group, the people who decide where a company's money and staff go, receives an AI request that has already picked its response. A team wants to buy a drafting assistant, build an agent, or automate a report. The recurring work has been named. So has the step that most limits what that work delivers, which this paper calls the constraint. The review is asked to say yes or no to the response the team chose. That is this paper's observation of how requests are often framed. No source counts how often.

This paper recommends comparing five responses before any is funded: automate the work, build a capability, buy a tool, hire or train people, or wait. Compare them on the same six questions. Set how much proof a response needs by how hard it would be to undo. Proof here means the evidence it must show before it is funded. Give waiting its own record: a trigger, an owner, and a review date. A trigger is a named event or measurement that brings the request back for a decision.

The stakes are money spent and, for some responses, a commitment nobody can leave. The order matters because of when the alternatives get weighed. Gartner's July 2024 release describes the sequence for a generative AI investment: if business outcomes meet or exceed expectations, expand it; if they fall short, explore alternative scenarios.[1] In that sequence the alternatives are weighed after the money is spent. That is this paper's reading of one analyst's description, not a finding.

Gartner has also forecast and reported how often projects stop after a proof of concept. In July 2024 it forecast that at least 30 percent of generative AI projects would be abandoned by the end of 2025, citing poor data quality, inadequate risk controls, escalating costs, and unclear business value.[1] In January 2026 a Gartner article reported that at least half had been, citing the same four causes.[2] One is a forecast and the other a later report. The article gives no method for the figure and says only that Gartner analyzed hundreds of implementations to name five failure points. Neither figure says how often a different choice would have helped.

RAND's interviews with 65 experienced data scientists and engineers also name missing data as one of five leading root causes of failed AI projects. They describe business leaders demanding that teams apply machine learning to problems that a few simple if-then rules could have solved.[3] Most interviewees were engineers rather than executives, so RAND says the results may lean toward leadership failures.

Each of Gartner's four causes is something a comparison of responses can ask about before money is spent: what data a response needs, what controls, what it costs to run, and what value it delivers. That is this paper's inference. Neither source says that comparing responses would have prevented the failures.

A companion paper on reviewing one AI request covers how to name the recurring work and its constraint, and the steps from request to approval. This paper covers the choice among responses once the constraint is named. It adds two things that paper leaves open: how hard each response is to undo, and when waiting is the right response. A third paper covers a review that holds several proposals competing for the same people and money. The parts depend on one another: the rule uses the five responses, the proof levels extend the rule's last question, and the section on waiting uses both, because waiting pays most when the alternative is hard to undo.

The five responses

The five are funding choices, and one piece of work can fit more than one. How hard each is to undo depends on its terms, so the notes on undoing below are this paper's judgments, not findings.

Automate fits work whose steps are stable, whose rules are known, and whose data is reliable, where the gain is speed, volume, or consistency. The automation may use AI, fixed rules, scripts, or workflow tools. Fund the simplest one that works reliably. RAND's interviewees describe teams told to use AI where simple rules would have done.[3] Afterward someone has to watch for drift, exceptions, and cost. A script or rule set is usually easy to switch off, though the old way of working can fade while it runs.

Build fits a capability that matters to strategy, depends on the organization's own data, or could become a lasting advantage. The organization then owns the design, the data pipelines, the testing, the support, and the future funding. Build only when owning and learning are worth those obligations. A build is hard to undo, because the money is spent and the team and the system must be kept or taken apart.

Buy fits a need that many organizations share, where a mature product already exists and speed matters more than custom technology. For agentic AI products, Gartner says many vendors relabel existing assistants, chatbots, and automation tools, and estimates that only about 130 of the thousands of vendors are real.[4] After the purchase the organization owns the setup, the integration, the training of users, data rights, and the way out. How hard a purchase is to undo depends on the contract

term, how deeply the tool is wired into other systems, and how much data and process have moved into it.

Hire or train fits when the constraint is judgment, domain knowledge, relationships, or the capacity to review. Hiring is partly irreversible because of the cost of hiring, training, and letting people go.[5] Training existing staff avoids the cost of letting people go, so it is easier to reverse.

Wait fits when the case is weak, the work is unstable, the data or controls are not ready, the vendor market is immature, or the organization cannot absorb the change. Waiting is the easiest response to undo, and it still has a price.

A change to the workflow is a response too. It counts under automate when it removes manual steps and under hire or train when it changes who does what. Some requests need a prerequisite first, such as cleaning data or settling who decides. That work is a separate request with its own owner and its own constraint, and the AI request waits for it.

How to choose among them

The comparison asks the same six questions of every response, not only the one the team prefers.

- Does it relieve the constraint, and how much of the gap between today's result (the baseline) and the result wanted could it close?
- Is what it needs in place: reliable data, a stable workflow, and people who can run it?
- What does it cost in total, counting licenses, usage, integration, review, support, and training, over the period the organization is committed to it? That period is the contract term, the life of a build, or the time it takes to hire, train, and later let people go.
- What can go wrong, and what controls does it need?
- Can the organization carry it without starving work that is worth more?
- How hard is it to undo?

The rule has three steps, and each needs the one before.

First, screen. Drop any response that cannot relieve the constraint. A response that needs something not yet in place, that the organization lacks the people or attention to carry, or that has a question with no answer goes to wait, with the missing item as

its trigger. A response that is hard to undo and lacks the proof its level calls for is replaced by a smaller version of the same idea that is easy to undo, and the smaller version goes through the screen. Record what result would justify the larger one. The next section sets the levels. A weak answer does not remove a response; it lowers its rank.

Second, rank what remains by how much of the constraint each would relieve each year, in the baseline's own measure such as days of delay or hours of rework, against its total cost for each year of commitment. Cost per year lets a three-year contract and a monthly plan sit side by side. If there is no baseline, measuring one comes first, as the companion paper on reviewing one AI request describes. The business owner, the leader accountable for the work's results, decides how much relief is worth how much cost, because that person lives with the result. Wait wins when no response remains, or when the best remaining response's relief is not worth its cost.

Third, settle ties. When two responses are close, as the business owner judges, choose the one that is easier to undo. When they are equally easy to undo, the business owner decides and writes down why.

A portfolio analyst, often in the project management office, puts all five responses on one page with the same six questions, so that no response is judged on a different test. A technical lead and someone who does the work every day supply the facts. The person who asked for the tool presents the request but does not judge it, because a response's advocate should not also be its judge. The companion paper on reviewing one AI request describes this group of four.

The cost and time are planning estimates, not measurements. An easy-to-undo request needs a short conversation among those people. A harder one needs a working session of about an hour, after the analyst and the technical lead have spent a few days gathering the facts. No source prices that time. The comparison pays when it costs less than undoing a wrong commitment.

How hard it is to undo sets how much proof a response needs

Pindyck treats the inability to undo an investment as a cost. His analysis of investment under uncertainty shows that an irreversible investment gives up the chance to wait for new information, and that the lost chance belongs in its cost.[5] The same paper says delay has a cost too, the risk that others move first or simply the benefit not received, and that the cost must be weighed against what waiting

teaches. This is theory about business investment in general, not a study of AI. It carries one idea here: how much proof a response needs should depend on what it would cost to reverse, not only on how large or exciting it is. This paper sets three levels, its own rule.

EXHIBIT 1

Three levels of proof, set by how hard a response is to undo

Easy to undo

Stops at the next review with little lost. Proof: a reason, a current measurement, a review date.

Costly to undo

Stopping leaves licenses, integrations, or retraining. Proof: a small test in real work.

Practically permanent

Multi-year contract, custom build, data moved, roles removed. Proof: the full case and a written way out.

Easy to undo means the response can be stopped at the next review with little lost beyond the time spent. An example is a trial by one small team using an approved tool. The proof asked for is light: the baseline, a stated reason, and a review date. A short conversation is enough.

Costly to undo means that stopping leaves paid-for licenses, integration work to unpick, or people to retrain. The proof is a small test in real work. Measure the result against the baseline before the response expands.

Practically permanent means a multi-year contract, a custom build the organization will own, sensitive data moved to a vendor, or roles removed and skills lost. The proof is the full operating case: real users, real data, working controls, the cost over the whole commitment, a named person responsible for the work after approval, and a written way out. NIST's AI Risk Management Framework lists a way out among its practices: procedures for decommissioning and phasing out an AI system, and for superseding, disengaging, or deactivating one whose results are inconsistent with its intended use.[6] It is voluntary guidance and does not say how much proof any purchase needs. At this level the steering group decides, not the business owner alone.

To place a response at a level, the business owner and whoever would carry the cost of undoing it, such as the finance or procurement lead, answer two questions. What

would going back cost, and how long would it take? What would be lost on the way: people, skills, data, or trust? If they cannot answer, treat the response as one level harder until they can.

The levels are not fixed to the responses. A bought tool on a monthly plan is easy to undo. The same tool bought for several years and wired into other systems is not. Before approving a hard-to-undo response, ask whether it can be made less permanent: a shorter term, an exit clause, a smaller first purchase, or keeping the old skills in the team while the new tool proves itself.

Waiting is a response with a price

A wait is a decision to take no new action on this request until a trigger occurs. Pindyck's reasoning is the case for it: when an investment is hard to undo and new information is coming, waiting keeps the option to act later, and acting too soon uses that option up.[5] The signs listed earlier are this paper's list of when waiting fits. Pindyck's reasoning applies most where something is expected to change, such as the data or the vendor market.

Waiting costs something. The benefit does not arrive, and the problem keeps costing what it costs now. Write that cost next to the trigger, in the baseline's own measure, per review period. Pindyck notes that strategic pressure sometimes makes it necessary to act before others do.[5]

A wait record has four parts.

- The trigger: an event or measurement that someone can observe, such as a data source reaching a stated level of coverage, a vendor product meeting a named requirement, or a risk decision being made.
- The wait's owner: the person who watches the trigger and brings the request back.
- The review date: the day the steering group looks again whether or not the trigger has occurred.
- The cost of the delay for each review period.

If the trigger has not occurred by the review date, the steering group extends the wait once with a written reason, revises the trigger, or stops the request. A wait with no trigger or no owner counts as a stop, recorded as one, and the request can return as a new request, because a delay with no condition for ending it cannot be told from neglect.

A worked example

Consider a proposal team that is missing deadlines because first drafts take too long to assemble. The team's leader is the business owner. This is an illustration, not a client account, and it reports no result. The request is to buy an AI drafting tool. The review runs the five responses through the screen before funding anything.

Automate: drafting is not stable, rule-based work. It depends on judgment about what each opportunity needs. Full automation would produce fast drafts that the team rewrites anyway. It is dropped.

Build: nothing in generic proposal drafting is different enough from what other firms do to justify owning a custom tool and its upkeep. A build would be hard to undo, and the team has no evidence at that level. The build is replaced by the Buy response already on the list, and a test of that response would show whether a build is ever justified.

Buy: a mature drafting tool could help, but only if the content behind it, such as past proposals, approved language, and current pricing, is organized and trustworthy. On a disorganized content base the tool speeds up the wrong starting point. The purchase lacks what it needs, so it goes to wait, with the content as its trigger.

Hire or train: a tool does not fix a shortage of people who can build a winning argument. Suppose that following recent proposals shows senior writers losing much of their time to assembling first drafts. Protecting that time then relieves the constraint, costs little, and is easy to undo. It remains.

Wait: the trigger for the purchase is a proposal library that reaches a stated level of coverage and freshness. The business owner is the wait's owner. The review date is the next quarterly planning meeting, and the record states what each month of slow drafts costs in missed deadlines.

Only one response remained, so ranking and tie-breaking did not run.

How hard the purchase is to undo changes the outcome. If the vendor offers a monthly plan, a trial on the proposal types whose content is already current is easy to undo and needs only a reason, a baseline, and a review date. If the vendor requires a multi-year contract and a link into the proposal system, the purchase is practically permanent, and the full operating case has to exist first.

The decision record might read: "Protect senior writers' time now. Fund the proposal library clean-up as its own request. Wait on the purchase until the library reaches the

stated coverage; owner, the team's leader; review at the next quarterly planning meeting."

Five ways the choice goes wrong

These five are separate. They are this paper's judgment, and only the first draws on a cited source.

The first is automation theater. The team automates a visible task because it can, then cannot show that a customer, process, or financial result improved. McKinsey says many organizations deploy AI in ways that are more visible than valuable.[7] The ranking step catches it, because a response must show how much of the constraint it relieves.

The second is prestige building. A team funds a custom solution because it looks strategic, when a mature vendor feature or a workflow change would relieve the constraint faster. The tie-break catches it, because the easier-to-undo response wins when two are close.

The third is procurement substitution. A tool is bought to avoid a harder decision about ownership, process, data quality, or review. The software arrives, and the constraint stays. The screen catches it by asking whether what the response needs is in place.

The fourth is headcount avoidance. AI is asked to cover a judgment, relationship, or change gap that needs skilled people, and the organization saves a role on paper and pays for it later in quality, trust, or adoption. Here the question about undoing a response catches it, because removed skills are slow to rebuild.

The fifth is passive waiting. Leaders defer a request without naming what would bring it back. The wait record catches it, and the rule that a wait without a trigger counts as a stop closes it.

Objections, and what the sources do not show

It adds process to every AI idea

The comparison scales with how hard a response is to undo. A trial at the easy level needs a short conversation, not a page. If most requests turn out to be easy to undo,

this comparison will rarely run. The cost is hours of a few people's time, against a multi-year commitment that nobody can leave.

Waiting can become drift

It can, and the wait record exists because of that. A trigger can also be set so far away that nothing ever returns. The review date is what forces a decision, because the steering group must look again whether or not the trigger has occurred. The rule that a wait without a trigger or an owner counts as a stop removes the comfortable version of waiting.

The rule is the paper's own

Where a business case already lists alternatives, this paper adds the levels of proof, the screen, and the wait record. The six questions, the three levels, and the order of screen, rank, and tie-break are this paper's design. Pindyck gives the reason to count undoing as a cost, and NIST lists exit procedures as practice. Neither tests the levels or the order. None of the sources compares building with buying, buying with hiring, or any response with waiting. A different organization may weigh the questions differently, which is why the business owner does the weighing. Vendors, models, and prices change quickly, so the review date matters.

What to do next

Take one AI request whose constraint has been named. If nobody can name it yet, start with the companion paper on reviewing one AI request. The portfolio analyst writes the five responses on one page, one line each, answering the same six questions. The business owner then marks each response's level, applies the screen, ranks what remains, settles any tie toward the response that is easier to undo, and states the proof the chosen response must show before it expands. If the choice is to wait, the page carries the trigger, the wait's owner, the review date, and what the delay costs. The business owner keeps the page as the decision record and updates it at each review.

Notes and sources

[1] Gartner, "Gartner Predicts 30% of Generative AI Projects Will Be Abandoned After Proof of Concept By End of 2025," July 29, 2024.

<https://www.gartner.com/en/newsroom/press-releases/2024-07-29-gartner->

[predicts-30-percent-of-generative-ai-projects-will-be-abandoned-after-proof-of-concept-by-end-of-2025](#)

[2] Arun Chandrasekaran, "Why 50% of GenAI Projects Fail — And How to Beat the Odds," Gartner, January 26, 2026. <https://www.gartner.com/en/articles/genai-project-failure>

[3] James Ryseff, Brandon F. De Bruhl, and Sydne J. Newberry, "The Root Causes of Failure for Artificial Intelligence Projects and How They Can Succeed: Avoiding the Anti-Patterns of AI," RAND Corporation, 2024.
https://www.rand.org/pubs/research_reports/RRA2680-1.html

[4] Gartner, "Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027," June 25, 2025. <https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027>

[5] Robert S. Pindyck, "Irreversibility, Uncertainty, and Investment," Journal of Economic Literature 29(3), 1991, 1110–1148; NBER Working Paper 3307, 1990.
<https://www.nber.org/papers/w3307>

[6] NIST, "AI Risk Management Framework Core," excerpt from AI RMF 1.0, 2023.
<https://airc.nist.gov/airmf-resources/airmf/5-sec-core/>

[7] Johannes-Tobias Lorenz, Joshan Cherian Abraham, Robert Levin, and Douglas Ziman, "From promise to impact: How companies can measure and realize the full value of AI," McKinsey & Company, April 24, 2026.
<https://www.mckinsey.com/capabilities/quantumblack/our-insights/from-promise-to-impact-how-companies-can-measure-and-realize-the-full-value-of-ai>

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani

This white paper may be shared freely with attribution. © 2026 Marco Policani.